Code-Command Research Agenda 1.0.0 – Alignment, Gaps, and Next-Build Priorities

Overview

The 100-item research agenda you drafted is strongly aligned with the existing Code-Command blueprint, invariant tags, and engine specifications that are already captured in your three core research artifacts. Those artifacts already define concrete behaviors for the 10 CC- tags, a WASM-based cc-engine, a GitHub-backed virtual filesystem, and custom token-walker validators, which together answer portions of many of the RQ/DR/DQ items. This report maps your agenda onto that existing design, identifies which questions are already answered at the spec level, which are partially covered, and which remain truly open research or implementation work.[1][2] [^3]

Existing foundations relevant to the agenda

Your earlier "Sovereign Coding Agent" invariant framework defines the 10 CC- tags (CC-VOL, CC-LANG, CC-CRATE, CC-FILE, CC-FULL, CC-DEEP, CC-ZERO, CC-PATH, CC-SOV, CC-NAV) with machine-checkable logic, YAML/ALN policy surfaces, and validator sketches. The file-structure blueprint then instantiates those tags into a concrete repository layout (index.html, coreengine src, js app, specs) with explicit destinations and responsibilities per file. The engine spec document (cc-engine1 and cc-vfs1) further defines the validator contract, cc-vfs snapshot schema, Single-Iteration Task Queue (SITQ), and CCTokenWalker semantics, including performance targets and cross-language behavior.[2][3] [^1]

Together, these documents already give you:

- A full definition of CC- tags and how to embed them in YAML frontmatter and FILE headers.[3][2]
- A WASM-export surface for validation, VFS operations, and SITQ execution with CC-ZERO constraints.[1][2]
- A two-tier validation model (Tier 1 string/byte scans, Tier 2 custom token walker) that addresses CC-FILE, CC-FULL, CC-LANG, CC-PATH, CC-DEEP, CC-SOV, CC-VOL, CC-CRATE, and CC-NAV.[3][1]
- A VFS and TaskQueue contract that channels all writes through validation and cc-vfs snapshots.[^1]

This means a substantial subset of your new research questions are already answered conceptually, and the next step is to translate them into concrete Rust/JS/CPP code and narrow implementation details.[2][1]

Mapping Research Questions (RQ-01 – RQ-34)

Directory discovery, VFS, and CC-NAV (RQ-01, RQ-02, RQ-19, RQ-23, RQ-34)

The invariant framework defines CC-NAV as requiring at least one custom directory-walking function implemented with std::fs (for native) or a VFS abstraction (for WASM), with a validator that detects such functions heuristically via the presence of readdir-like calls and recursion. The file-structure blueprint assigns this responsibility to coreengine/src/navigator.rs, which exposes a walkdir(path: &Path) -> Vec<PathBuf> function that uses std::fs::read_dir and recursion when compiled natively, and VFS list operations in WASM. For discovery of .rs, .js, .cpp, .h, .aln, and .md files to a depth of 3, this function can accept a maximum-depth parameter and terminate recursion when the path component count reaches the limit, leveraging the same normalization logic used by CC-PATH and CC-DEEP.[2][3] [^1]

Your engine spec already defines a VFS snapshot schema (cc-vfs1) that holds path, content, sha, and isdir flags for all entries, which can serve as the in-memory representation for directory trees and as the persistence format for session state. For mounting/unmounting virtual workspaces, you can treat a cc-vfs snapshot as the mount unit and store it as a JSON/ALN document, with mounting meaning ccinitvfs(snapshot_json) and unmounting meaning discarding or archiving that snapshot without touching the underlying GitHub repository. RQ-34 (session state) can build directly on this by extending the snapshot to include open file list, editor cursor positions, and task history as additional fields, serialized in the same JSON envelope.[3][1]

ALN/Markdown policies and parsing (RQ-03, RQ-06, RQ-29, RQ-31, RQ-32)

Your invariants and engine spec already treat ALN (Anchor-Like Notation) as a policy language layered on top of Markdown, with YAML frontmatter, ALN blocks, and plain-text sections describing tag semantics and validator contracts. The cc-policy.aln file is designated as the source of truth for all CC- tags, with a frontmatter listing the active tags and per-tag sections providing natural-language intent, machine rules, and pseudocode. The spec also outlines a lightweight ALN/Markdown policy parser that can extract cc-invariants, tags, and per-tag definitions without relying on external crates, using simple line-based scanning and frontmatter parsing.[1][2] [^3]

For RQ-03 and RQ-31, this means ALN is effectively a Markdown superset or sidecar format where:

- YAML frontmatter defines global policy metadata (cc-invariants, status, stack, etc.).
- ALN headings and key/value lines describe tags and contracts in a regular, line-oriented schema that can be parsed with a custom token walker.[2][3]
  A minimal recursive-descent parser for a subset of ALN can be implemented with standard Rust features—enums for node types, a scanner for tokens like identifiers, tags, and colon separators—without macros or external parsers, matching your RQ-06 constraint. RQ-29 is addressed by binding ALN policy statements to specific checkcc_*

functions (e.g., checkccfull, checkccpath) and then invoking those from the validator; your spec already defines such functions and their contracts.[3][1]

Custom JS engine subset and cross-language behavior (RQ-04, RQ-17, RQ-18, RQ-27)

The existing blueprint assumes a WASM-based Rust core that exports a small CC-API and a JavaScript front-end that instantiates the WASM module, renders the editor and file tree, and calls into the engine for validation and VFS operations. It does not yet define a full custom JavaScript engine, but the invariant documents describe cc-token-walker as language-aware and cross-language, with a LanguageHint enum used to adapt pattern sets across Rust, JS, and C++. This provides part of the answer to RQ-18: a symbol table can map command names to function identifiers inside Rust, while LanguageHint and ScanProfile allow reuse of the same scanner across languages with minimal branching.[1][2] [^3]

RQ-04 and RQ-17—embedding JavaScript utilities and interpreting them—are not concretely specified yet, but the architecture favors a minimal engine that supports only the constructs required to run configuration/policy logic and lightweight automations. A reasonable path is to start with a very small, hand-rolled ES5 subset: var, function declarations, if/else, for loops, simple objects, and JSON.parse/JSON.stringify, as you already listed in your DR-10 definition request; the engine can compile these into a custom bytecode representation at startup and interpret them in Rust, without JIT or GC, as suggested by the objection response that advocates an AST-walking interpreter. RQ-27's asynchronous task queue can be approximated with a simple event loop over a vector of JS tasks plus std::thread::park/unpark in native mode and a setTimeout-style cooperative loop in the browser, with the Rust side tracking ready tasks and scheduling their execution.[3][1]

Rust–C++ interaction and custom C components (RQ-05, RQ-28)

Your file-structure blueprint already includes a C++ helper at cpp/fallback/path-sanitizer.cpp that implements path normalization in C for performance or integration reasons, invoked from Rust when needed. The invariants docs emphasize that C and Rust implementations must match the same validator and VFS contracts (cc-engine1 and cc-token-walker-1), but they do not yet specify the exact FFI wiring. RQ-05 can be addressed with a standard extern "C" interface from Rust and manually linked object files, compiled ahead of time for the GitHub-hosted agent or embedded as static objects in your WASM bundle when targeting the browser.[2][1]

For RQ-28 (file watching in C++), none of the artifacts define a portable file-watching mechanism; given your no-external-sources rule and the GitHub-as-VFS design, the primary change notification path is via GitHub API calls and cc-vfs snapshots rather than native file watchers. That suggests deferring C++-level file watching and instead polling GitHub trees or receiving updated snapshots through the host environment, aligning better with CC-ZERO and CC-SOV.[2][3]

Environment detection and GitHub Action context (RQ-07, RQ-33)

The invariants and file-structure blueprint explicitly include a .github/workflows/cc-invariants.yml workflow that runs the cc-engine validator on pull requests and blocks merges when tags fail. Your engine spec notes that the WASM module must not access environment variables directly under CC-ZERO, and that all state must flow through the cc-vfs and cc-API calls. RQ-07 and RQ-33 can therefore be answered by having the host environment (browser UI versus GitHub Action runner) pass a profile field (github, local, memory-only) to the engine via the SITQ task payload and VFS initialization, rather than having the engine inspect environment variables. The GitHub Action context is explicitly known to the workflow YAML and Node (or JS) harness, which can configure the engine to run in "lint-only" mode (no writes) by selecting the github profile and omitting writefile tasks in ccexecutetask payloads.[1][3] [^2]

Deterministic function identifiers and CC-VOL (RQ-08, RQ-25)

The invariants research already defines CC-VOL as an assertion that an artifact contains at least N function-like declarations, with a validator that uses cc-token-walker to count fn, struct, function, and class patterns per file. For RQ-08 and RQ-25, the same token walker can compute a deterministic identifier per function by combining a stable token stream fingerprint (e.g., normalized function name, parameter count, and first few tokens of the body) into a small rolling hash or structural fingerprint that avoids full cryptographic hashing. The engine spec's emphasis on zero-alloc scanning and O(N) time over file length supports this approach, and function identifiers can be stored in the per-file symbol table that CC-CRATE uses to detect new code constructs between snapshots.[3][1]

Line endings, path integrity, and CC-PATH (RQ-09, RQ-15, RQ-23)

CC-PATH and CC-DEEP are already defined with explicit path normalization rules: reject backslashes, reject internal double slashes, and ensure depth(path) ≥ 3 for core modules. The validator pseudocode uses byte-level scanning for "//" and "\" and relies on normalize_path to remove empty segments and resolve ".." without escaping the repository root. RQ-15's finite-state machine for path integrity can be implemented directly from this pseudocode by treating each path string as a stream of characters, tracking state across separators, and maintaining a depth counter.[1][3]

For RQ-09 (line endings), the engine spec and token-walker research emphasize operating on UTF-8 text and line-based iteration, but they do not prescribe a specific normalization strategy for \n versus \r\n. A straightforward approach is to normalize CRLF to LF in memory for scanning purposes while preserving original content in the cc-vfs snapshot for hashing; this can be modeled as a logical view over the same underlying bytes, where the scanner sees lines with \n while the stored sha corresponds to the unmodified GitHub content.[3][1]

Dynamic struct insertion and interpreter-like execution (RQ-10, RQ-21)

The cc-engine spec describes the validator and TaskQueue as pure functions over code strings and cc-vfs snapshots, not as a dynamic interpreter that accepts new Rust structs at runtime. RQ-10's request for an in-memory structure that allows dynamic insertion of new struct definitions without recompilation is not directly addressed, but the spec suggests a path: treat struct and function declarations as entries in a language-neutral symbol table, populated by CCTokenWalker scans, rather than as compiled Rust types. The TaskQueue then operates over these symbols and code strings via policies, not via dynamically instantiated Rust types.[^1]

For RQ-21's topological ordering of module dependencies, the engine spec offers guidance in the form of cc-vfs snapshots and a logical representation of files and imports; a simple topological sort can be implemented using Vec and HashMap keyed by module names and import relationships discovered by cc-token-walker, without external graph crates.[3][1]

CC-LANG, CC-FILE, CC-FULL, and CC-DEEP automation (RQ-11, RQ-14, RQ-16, RQ-20, RQ-24, RQ-26, RQ-30)

Several of your RQs are already answered in detail by the invariant framework:

- CC-LANG (RQ-11) is enforced by restricting extensions to .rs, .js, .cpp, .h, .aln, and .md, with the validator rejecting others and optionally scanning content for foreign syntax.[2][3]
- CC-FILE (RQ-24) is enforced by scanning the first N lines for a FILE header (e.g., "FILE ./src/core/engine.rs" or "") and extracting the path; every write operation must include a valid header, and the path is validated against CC-PATH and CC-DEEP before any file is persisted.[2][3]
- CC-FULL (RQ-16) is enforced by a simple substring search for "...", "rest of code", and "omitted" within code blocks, rejecting any artifact that contains placeholder markers. [^3]
- CC-DEEP (RQ-19, RQ-26) uses a normalized path and depth counter to verify that core modules reach at least three path segments; the invariant spec also calls out special handling for core versus leaf files, so that leaf utilities may live at shallower depths.[2][3]

RQ-14's minimal JSON builder is partially answered by the engine and validator code sketches, which show a JsonBuilder-like struct implemented with a push-down automaton approach, assembling JSON strings with explicit start_object, key, value, and end_object operations, avoiding ad-hoc string concatenation and external serde_json dependencies. RQ-20's environment variable and CC-ZERO constraints are captured in the spec's requirement that the WASM engine not access environment variables or perform setup operations; configuration is instead passed through cc-vfs snapshots and task payloads.[1][2] [^3]

RQ-26's binary representation of the 10 CC- tags is not specified, but the existing documents suggest representing them as short strings in YAML frontmatter and ALN, and as small enums or bitfields in Rust for fast runtime checks; this is consistent with the ScanProfile and ScanMask concepts already outlined for cc-token-walker.[1][3]

RQ-30's Blacklist(*/) detection is defined in the objections section: only the exact pattern "(/)*" (with parentheses) is treated as contamination, not generic comment terminators like "*/"; the agent must detect such patterns context-sensitively and emit a professional report. The engine spec does not yet define the exact report format, but your research docs describe a validation result schema with tag, message, path, line, and column fields, which can be reused to report Blacklist violations.[3][1]

Memory limits, allocators, and heap ceilings (RQ-31)

The cc-engine spec and surrounding research emphasize low-overhead, O(N) scanning and bounded working sets, but they do not define a custom memory allocator. However, your separate Data_Lake and ccvfs research introduces a WorkingSet controller and a CcVfs trait that sits on top of a BlockStore, with explicit working-set ceilings derived from real browser, GitHub, and object-store limits. Translating that into a custom allocator for Code-Command can be achieved by implementing a simple bump-or region allocator that enforces a maximum heap size for cc-engine operations and denies scans or writes that exceed this limit, logging such events through the TaskQueue.[1][3]

Mapping Definition Requests (DR-01 – DR-33)

CC-FILE syntax, headers, and file destinations (DR-01, DR-17)

The invariant policy and engine spec define CC-FILE as requiring a header line within the first N lines of a file, using a language-appropriate comment syntax and the literal marker FILE followed by a normalized relative path. Examples include "FILE ./core/engine/src/lib.rs" in Rust and "" in HTML; the path must be non-empty and conform to CC-PATH and CC-DEEP rules. The validator's checkccfile function is responsible for extracting the path, and write operations are blocked if the header is missing or inconsistent with the ValidationRequest path.[2][3] [^1]

This provides a precise byte-level syntax: comment prefix + optional whitespace + "FILE" + whitespace + path + comment suffix, with colon placement avoided to keep the parser simple; path normalization then occurs in Rust using a custom PathBuf-like function rather than OS-level utilities.[2][3]

Function-structs, CC-CRATE, and task-behavior (DR-02, DR-26, DR-27, DR-28)

Your research artifacts define CC-CRATE as requiring each delta or iteration to introduce at least one new code construct (struct, fn, mod, class), with the validator comparing current and previous symbol sets to detect fresh logic. A "function-struct" can therefore be modeled as a Rust struct with associated impl methods representing a unit of new logic; your cc-token-walker and engine spec also define a Symbol representation that includes name and kind, making it straightforward to capture both data and behavior in an internal struct.[3][1]

"Task-behavior" is defined through the SITQ contract: tasks are JSON objects with kind, path, content, sha, and tags fields; a "task" is a high-level operation such as writefile, deletefile, or validateonly, and "task-behavior" is the deterministic mapping from cc-vfs snapshots and task

queues to new snapshots and validation reports. CC-VOL encodes the requirement for a minimum number of distinct functions or code blocks per response or artifact; the engine spec leaves the threshold V_min to invariants.aln, where it can be set as a policy constant.[1][2]

ALN/Markdown, policies, and machine-readable tags (DR-03, DR-07, DR-16, DR-21, DR-24, DR-25, DR-31)

ALN/Markdown is already treated as a hybrid format in your policy files: YAML frontmatter declares cc-invariants, status, stack, and tag state, while ALN sections define semantics in a key/value style that is easy to parse with a custom token walker. Policies are split into mandatory invariants (hard rules like CC-FILE, CC-FULL, CC-SOV) and advisory guidelines; the former are expressed as CC- tags with machine rules and validator pseudocode, while the latter can be encoded as non-blocking tags or comments in ALN.[2][3]

Machine-readable tags are represented in frontmatter as cc-invariants arrays and in code as CC- prefixed identifiers; the engine spec defines them as enums and bitfields inside the validator, with ScanProfile and ScanMask mapping tag sets to token-walker pattern sets. A "research-data" object can be modeled as a structured record containing cc-vfs snapshots, token-walker metrics (e.g., line counts, function counts, path anomalies), and validation outcomes; this is consistent with the engine's ValidationResult schema and the test corpus described for cross-language conformance.[3][1]

The YAML frontmatter format for tags is already specified in cc-policy.aln, with fields like cc-invariants, kind, version, status, and stack; this can be extended to additional per-file overrides or profile flags without changing the core parser.[2][3]

Zero-setup, rate-limiting, cloud-grade performance, and connectors (DR-04, DR-06, DR-09, DR-13, DR-18, DR-19, DR-20, DR-22)

CC-ZERO is defined as "No-Init Boot": entrypoints must contain only relative imports and no references to std::env::temp_dir, install, or setup functions; the engine spec further requires that the WASM module not access environment variables or OS-level filesystems directly. "Zero-setup" thus becomes measurable: any use of environment variables, installation commands, or temp directories constitutes a violation, and validators are explicitly instructed to scan entry files for banned tokens like install, setup, and env::temp_dir.[1][3]

The "instantly-loadable" requirement is implemented by a single index.html file that loads the JS app, WASM engine, and UI components, requiring only a GitHub Pages or static HTML hosting environment; your blueprint specifies this in detail, including the containers for editor, file tree, and terminal, and the script tags for main.js and monaco loader. "Rate-limiting" is acknowledged by the use of an IndexedDB cache for GitHub data and by avoiding background polling; all GitHub interaction is mediated through explicit cc-vfs tasks and user actions, which keeps request volume under control.[3][2]

"Cloud-grade performance" is partially quantified by the cc-token-walker benchmark goal: scan a synthetic 10,000-line Rust file in under 5 ms in a browser WASM environment, with throughput metrics defined in terms of lines per millisecond and bytes per millisecond. Additional performance metrics like memory footprint can be derived from the WorkingSet

ceilings in your Data_Lake/ccvfs research, which uses environment-specific capacity envelopes.[^1]

Connectors are not fully formalized in the existing documents, but the engine spec and JS architecture imply that a connector is a thin layer that maps an AI-Chat platform's message format and repository access into cc-vfs snapshots and SITQ tasks, then returns ValidationResult and task reports back to the platform; the lifecycle would be init (load repo -> ccinitvfs), run (ccexecutetask per prompt), and shutdown (persist snapshots or discard).[2][1]

Sovereign stack, no externals, wiring, and interoperability (DR-10, DR-11, DR-12, DR-13, DR-20, DR-23, DR-30, DR-32, DR-33)

The invariant documents explicitly define the sovereign language stack as Rust, JavaScript, C/C++, and ALN/Markdown for policies, with CC-LANG enforcing extensions and CC-SOV enforcing allowed import sources. "Sovereign" in this context means no third-party crates like serde_json, reqwest, or tree-sitter; only the Rust standard library and minimal WASM toolchain glue are permitted. This is reinforced by the empty dependencies section in coreengine/Cargo.toml and the explicit ban lists in checkccsov pseudocode.[3][1] [^2]

"Wiring" is described as the explicit function-level connections between modules: lib.rs exports cc-API functions and delegates to validator.rs, vfs.rs, navigator.rs, and taskqueue.rs; JS app/main.js initializes the WASM engine and hooks UI events to these functions; js/app/github-api.js maps GitHub API responses into cc-vfs snapshots; and cpp/fallback/path-sanitizer.cpp can be called from Rust for path normalization. Interoperable logic across Rust, JS, and C++ is achieved by using language-neutral contracts like cc-vfs snapshots, task JSON payloads, and ValidationResult objects, with type-specific marshalling handled in a small number of boundary modules.[1][2]

The directory-navigation state machine required by CC-NAV is described in your invariants as a custom function that performs directory walking via std::fs::read_dir or cc-vfs list operations, potentially using recursion and a depth counter; this function, along with CCTokenWalker and TaskQueue, forms the "discovery logic" core where research-objects, pathfinding logic, and file placement strategies are implemented. Error handling across the engine is modeled as returning ValidationResult and TaskQueue reports rather than panicking; the engine spec requires determinism and well-structured error messages with tag, message, and location fields.[3][1]

Mapping Detail Queries and Objections (DQ-01 – DQ-23, OI-01 – OI-10)

The detail queries largely request algorithmic or sequence-level descriptions for capabilities already outlined in your research; your invariants and engine spec provide most of the components:

- DQ-01 and DQ-02 follow directly from navigator.rs and cc-vfs: std::fs::read_dir based BFS/DFS with depth checks and a DeepWalker iterator that wraps this logic.[2][3]
- DQ-03 and DQ-19 (JS expecting fs or DOM/window) are addressed by the GitHub-as-VFS design and by treating DOM-dependent JS as unsupported in the AI-Chat/browser

environment; the agent can detect require('fs') or window references via CC-SOV and treat them as violations.[^3]

- DQ-04 and DQ-11 (VolumeLock struct and JsonBuilder) are answered by the scan profiles and minimal JSON builder sketches: VolumeLock tracks counts of function-like constructs, while JsonBuilder constructs JSON strings via explicit state transitions.[1][3]
- DQ-05 and DQ-15 (CC-DEEP validation and CC-NAV flowchart) are implemented via the normalize_path + depth counter approach and a walkdir that groups files into research objects based on extension and tag requirements.[1][3]
- DQ-08 and DQ-16 (CC-SOV and CC-FULL checks) use line scans and substring detection for forbidden crates and placeholder markers.[^3]
- DQ-21 (YAML frontmatter validation) is handled by the ALN/Markdown policy parser, which reads frontmatter until '---', extracts cc-invariants and tag state, and ignores unknown keys for forward compatibility.[2][3]
- DQ-22 (state persistence across prompts) maps to cc-vfs snapshots and session-state extensions, serialized as JSON/ALN files or stored in the host environment's storage (IndexedDB, object stores, or platform-specific session storage).[^1]
- DQ-23 (building a new connector) follows the connector lifecycle described earlier: prepare cc-vfs snapshot, construct SITQ tasks from platform prompts, call ccexecutetask, and surface ValidationResult and reports back to the platform UI.[2][1]

Your objection identifiers and counter-strategies (OI-01 – OI-10) are already reflected in the research: custom JsonBuilder instead of serde_json, minimal JS engine instead of full JavaScript interpreter, .cc-cache directory instead of temp_dir, use of index and arena allocation instead of borrow-tangled references, no network calls beyond GitHub, core vs leaf module distinction for CC-DEEP, context-sensitive Blacklist(*/) detection, lowest-common-denominator JS connector interface, CC-FILE headers as machine-readable comments, and explicit allowance of the Rust standard library and WASM toolchain.[2][3] [^1]

Next-build priorities unlocked by this agenda

The research agenda is not a blank slate; it compresses and sharpens a set of design questions that your existing invariants, file structure, and engine specs already partially answer. The most impactful next steps are therefore implementation-oriented:

1. Finalize and implement cc-token-walker with ScanProfile and LanguageHint, meeting the 10,000-line, 5 ms WASM benchmark and fully enforcing CC-VOL, CC-CRATE, CC-SOV, and CC-NAV.
2. Implement coreengine/src/validator.rs and wire it to cc-policy.aln and specs/invariants.aln so all tag semantics live in one policy surface but execute via hardcoded Rust functions.
3. Finish coreengine/src/vfs.rs and js/app/github-api.js so cc-vfs1 snapshots and SITQ tasks can be exercised end-to-end against real GitHub repositories from index.html.
4. Implement js/app/main.js, js/app/editor/file-tree.js, js/app/terminal/output-panel.js, and the minimal UI shell described in your blueprint, ensuring CC-ZERO and CC-DEEP compliance in file paths and structures.

5. Add a Blacklist(*/) detector and professional contamination report generator to the validator, integrating it with ValidationResult and TaskQueue outputs.
6. Define and codify V_min (CC-VOL threshold), core-module path patterns for CC-DEEP, and banned dependency lists for CC-SOV in invariants.aln so they can be tuned without changing Rust code.

Once these pieces are implemented, many of your research questions move from speculative to resolved, and Code-Command becomes a governed, zero-setup, GitHub-native coding agent whose behavior is constrained by machine-readable policies you fully control.[3][1] [^2]

References

1. this-research-focuses-on-creat-b00MzfuQS32A1llas2CIQA.md - The research docs already define explicit ProofofFunctionality PoF styles for cc-token-walker and cl...
2. based-on-the-research-synthesi-GpW30ievRYyoRNZb49h_9Q.md - img srchttpsr2cdn.perplexity.aipplx-full-logo-primary-dark402x.png styleheight64pxmargin-right32px
3. invariants-8Xz_1YHRT1W1xd_tCBpMyA.md - img srchttpsr2cdn.perplexity.aipplx-full-logo-primary-dark402x.png styleheight64pxmargin-right32px